

Axel Estrada

5/12/26

CMPU-395: Machine Learning

Professor Stapleton

## **Final Project Final Draft**

### **Introduction and Motivation**

Audio classification is a huge field within machine learning. Applications range from reading sheet music to generating digital audio effects to classifying instruments by various factors. In the guitar world, there are endless combinations of effects a person can use to produce a desired sound and feel. Many amplifier and effects models have been developed through both analog and digital means, and much of the machine-learning work on guitars has focused on how to digitally recreate analog amp and effect models. While these models have gotten so good that they're nearly indistinguishable from their physical counterparts (Bognár 2), not as much work has been done on audio classification of guitar effects.

Classification differs from effects recreation not only in its goal, but in its complexity. The major difference is that classification requires extensive data on various combinations of individual amplifiers, effects, and guitar tones, as well as on combinations with other instruments and guitars played at the same time. Research on this topic has been conducted before, both in the context of individual guitar recordings and in mixed-instrument recordings (Hinrichs et al., 2022), though it remains quite scarce. Previous research has also been done to create a machine-learning model that can classify electric guitar recordings from different manufacturers and models (de Castro Rabelo Profeta and Schuller 478), with varying degrees of success.

This project is an exploratory analysis for creating and training a machine learning model that can identify electric guitar pedal effects from audio recordings. The guiding research questions for this project were as follows:

1. Can we develop a model that accurately predicts if an electric guitar audio recording is using overdrive, distortion, or fuzz audio effects?

## 2. What Machine Learning models work best for guitar audio classification?

Much of the literature focused on these types of questions utilizes Mel-spectrograms for training their models. Mel-spectrograms are a useful visualization tool for seeing what audio sounds like. It is a time-frequency representation of an audio signal in which the frequency axis is scaled according to how humans generally perceive sound (Roberts, 2020). Humans have an easier time distinguishing between lower frequencies than they do higher frequencies. As such, mel-specs compresses higher frequencies while emphasizing lower frequencies.

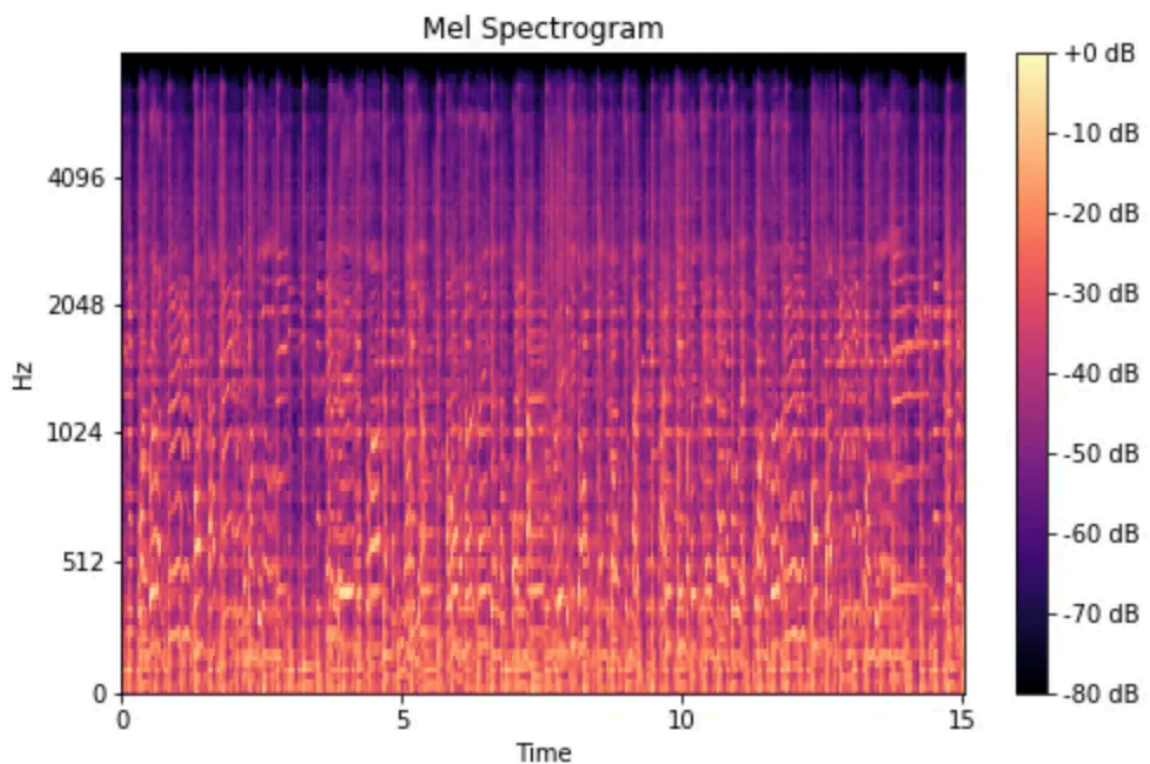


Figure 1: Example of a Mel-Spectrogram (Roberts, 2020)

The initial expectations for this project were that Convolutional Neural Networks would serve as a great starting point for answering these questions. This is because mel spectrograms are a visual representation of the different frequencies present in an audio recording, so in theory, you could have different blocks and filters trained to capture different properties or styles within the audio. Various papers focusing on similar research questions used CNNs trained on labeled mel-spectrogram representations of audio recordings. Previous research by

Hinrichs et al. managed to create a model that achieved up to 97.4% accuracy (Hinrichs et al. 1) for classifying guitar effects while other instruments were playing.

## **Data Description**

The GUITAR-FX-DIST (GFD) and IDMT-SMT-Guitar (ISG) Datasets were the primary datasets used for this project. ISG is the largest publicly available data set of electric guitar and bass recordings. The dataset consists of 55044 WAV files (44.1 kHz, 16-bit, mono), with 20592 monophonic guitar notes and 13860 polyphonic guitar sounds. Monophonic audio samples are those that only contain one discrete note at a time. Polyphonic audio samples contain multiple notes at the same time.

ISG guitar recordings are organized into four subsets. The first subset contains different playing techniques, including plucking styles and expression styles. The second subset consists of 400 monophonic and polyphonic note events, each played with two different guitars, with no expression styles. The third subset is five short monophonic and polyphonic guitar recordings. The last subset is 64 short musical pieces grouped by genre.

The GFD dataset is an extension of the ISG dataset created by Marco Comunità. It used the same recordings as ISG but processed with overdrive, distortion, and fuzz audio effects. A total of 14 different pedals were used, and each recording was produced through a MATLAB script he wrote. The dataset is organized into four sets: monophonic discrete, monophonic continuous, polyphonic discrete, and polyphonic continuous. Discrete and continuous describe how the audio samples change over time (i.e., in steps or smooth curves). Each set contained folders with the audio files, organized by the pedal that was used to generate them. Within each folder was a .csv of the parameters used to generate the effects for that sample. The dataset contains roughly 550k samples (~305 hours) and also 550k mel-specs, and utilizes the same audio format as the ISG dataset. The mel-specs were organized the same as the audio recordings.

Most of my exploration was done with the GFD dataset, as it provided both the audio and mel-specs recordings necessary for training the models, simplifying a large part of the early analysis. The original audio and spectrogram files were not modified throughout the research,

so as to ensure training and evaluation consistency. The dataset was remapped using the following pedal-to-effects mapping:

**Table 1: Effects-Pedal Mapping**

| Effect     | Pedals                       |
|------------|------------------------------|
| OverDrive  | 808, TS9, BD2, OD1, SD1, MGS |
| Distortion | DS1, RAT, DPL, MT2           |
| Fuzz       | FFC, BMF, RBM, VTB           |

All mel-spec folders were reorganized into HDF5 files so that data loading and processing could operate as efficiently as possible. HDF5 stands for ‘The Hierarchical Data Format version 5’. It is an open-source compressed file format meant for large, complex heterogeneous data, which uses a ‘file-directory’ like structure (Wasser, 2026). Within this project, it allowed me to generate ‘shard’ files with long lists of mel-specs and their corresponding effects labelings, which could then be uploaded to Google Colab. Instead of needing to upload thousands of files and then access each one individually, we could upload a few organized files and access each spectrogram/label via list indexing. Reasons why this was necessary are explained in the methods section.

The final shard set used across all training and testing consisted of 6 groupings (Mono\_Continuous, Mono\_Discrete, Poly\_Continuous, Poly\_Discrete, Mono\_Clean, Poly\_Clean). Each grouping contained 4-6 shards, with roughly 10,000 samples per shard, except for the last shard, which would be anything less than 10,000 depending on how much was left. Both Mono\_Clean and Poly\_Clean contained significantly fewer samples than the other categories, since these are just the base audio recordings all other samples were generated from, and as such were forcibly spread across 5 shards each, so that train/test splits could be properly generated.

## Methods

The code for this project was written in Python as a notebook meant for Google Colab and Jupyter Notebook. Colab was used for two main reasons. The first reason was that the pro version offers access to a variety of different GPU models designed for machine learning training, which would make our training far faster than if we used just a CPU. The second reason was that I could remotely train models without affecting my computer's performance for other tasks.

A majority of this project was spent figuring out how to systematically and efficiently store, organize, and access all of the mel-spec files to then train a model on Colab. The initial idea was to take all of the GFD mel-specs, upload them to Colab, and then perform analysis from there. Unfortunately, this proved to be a far more difficult task than initially thought. The first major issue to arise was that Colab doesn't allow folders to be uploaded directly, only files. This was a problem 1) because of the sheer number of files we had (~ 516,000 samples), and 2) because if a session is left idle for too long, Colab erases all uploaded files, and the process needs to be done again.

The first solution I came up with was to upload all the files to GitHub and then download them remotely. This was immediately shut down after finding out that GitHub only allows a maximum storage of ~2 GB, and GitHub's large file storage (LFS) didn't extend that by much more. The next solution from there was to download the Google Drive Desktop app, and then synchronize the file folders from my local computer to my Google Drive. This process took forever, depending on my available internet connection, but eventually I was able to upload all of the original mel-specs.

Unfortunately, this then led to the next major issue. Google Drive I/O reads and writes are really, really slow. As such, parsing through thousands of files to train one model for even 10 epochs proved difficult, and given that we needed a stable connection the entire time (otherwise we'd lose the model), this proved highly impractical. I thought to copy the files from the synchronized folder to the '/contents' folder of the Colab notebook so that the file access speeds could be improved, but it also took a long time to perform, and ran into the previous issue of needing to do this every time the session is lost.

This is where the HDF5 files mentioned earlier came in. This file format allowed me to shrink the number of files needed to upload to Colab from ~516,000 down to ~60, depending on how many mel-specs are stored in each file, and also slightly decreased the dataset size (~25 GB down to ~19 GB). This significantly improved the processing speed of the two previous solutions and finally ensured a consistent way to upload our mel-spec dataset to Colab.

At this time, I also lost access to most of the high-end GPUs available for training (ran out of Colab training tokens), and the available GPUs were either on par with or slower than my laptop GPU. Even with this new process, however, training still takes quite a while, and the risk of losing our progress due to network issues was ever high due to my needing to move around a lot across campus. As such, I decided the best time to train was at night, when the model could use all of my system's resources.

Due to this series of unfortunate events, not much time could be spent training and testing different model architectures for the task. Two models were trained and evaluated. The first was a 'Simple\_VGG' neural network with 4 convolutional blocks, including a ReLU activation function and a Maxpooling layer at each block. At the end, an adaptive average pooling layer was applied in the off-chance that one of the mel-specs was badly formatted, followed by a flattening layer to map onto our categories. The Simple\_VGG model structure was kept the same throughout all training and testing sessions.

The other model I explored was a pretrained model called ResNet-18 available through the torchvision library, which utilized the ResNet architecture. This architecture features 18 residual blocks, each consisting of two convolutional layers, a batch normalization function, and a ReLU activation function. The key feature of this architecture is the skip connections at each block, which bypass one or more layers (GeeksforGeeks, 2025). The model was modified to work with our dataset and categories, so the first convolutional layer was changed from a 3-channel input to a single channel, and the final layer mapped onto our labels.

Two separate testing methods were implemented. The initial method involved training each model for up to 30 epochs and tracking the overall accuracy of the model across all epochs. Each model utilized Adam as its optimizer and Cross-Entropy-Loss for its loss function. Training and testing were done with a batch size of 128 samples and a learning rate of 0.001.

The test splits used were 80/20 or 90/10, with samples being randomly taken across each of the groupings for an even training distribution. The splits were done across shards and not the number of mel-spectrograms within each shard. Both models were trained on different train/test splits. The switch to a 90/10 split was an arbitrary choice to compensate for potential overfitting noticed during the Simple\_VGG training.

After some experimentation, I decided to develop a new training method that was more robust. This method involved implementing a k-fold cross-validation algorithm for training and testing the models. During training, this algorithm would keep track of both loss and validation accuracy across epochs, so that I could better understand how the model was doing over time. These values would then be stored onto a CSV file, which we could then graph the results of in a separately written function.

I performed 2 separate rounds of k-fold training/testing for each model architecture, resetting the models after each round. Simple\_VGG was kept the same as before, but the ResNet-18 model was switched from pretrained to not pretrained. This was done to further improve the model's interpretability, since we would know what it was being trained on. Each fold was trained for 10 epochs. The same train/test split was used for both models across each round, ensuring that I could better compare the models against each other. Test splits were 80/20 for all rounds, with the same batch size, learning rate, optimizer, and loss function as in the first train/test method.

## **Results**

From the training and testing I was able to run, I achieved the following results:

**Table 2: Training results for VGG and ResNet-18 (No k-folds)**

| Model Architecture             | Total Epochs | Test/Split | Total Training Time (min.) | Total Model Params. | Overall Accuracy |
|--------------------------------|--------------|------------|----------------------------|---------------------|------------------|
| Simple VGG (round 1)           | 10           | 80/20      | 71                         | 625,294             | 94.91%           |
| Simple_VGG (round 2)           | 20           | 80/20      | 133                        | 625,294             | 94.96%           |
| Simple_VGG (round 3)           | 30           | 80/20      | 213                        | 625,294             | 94.94%           |
| ResNet-18_Pretrained (round 1) | 10           | 90/10      | 74                         | 11,171,779          | 94.66%           |
| ResNet-18_Pretrained (round 2) | 20           | 90/10      | 148                        | 11,171,779          | 99.3%            |

**Table 3: Training results for Simple\_VGG and ResNet-18 (k-folds; k=5)**

| Model Architecture                | Simple_VGG (round 1) | ResNet-18 (round 1) | Simple_VGG (round 2) | ResNet-18 (round 2) |
|-----------------------------------|----------------------|---------------------|----------------------|---------------------|
| <b>Fold 1 Avg. Accuracy</b>       | 95.31%               | 96.22%              | 86.62%               | 92.69%              |
| <b>Fold 2 Avg. Accuracy</b>       | 84.82%               | 85.68%              | 96.50%               | 96.69%              |
| <b>Fold 3 Avg. Accuracy</b>       | 96.15%               | 96.26%              | 80.73%               | 80.48%              |
| <b>Fold 4 Avg. Accuracy</b>       | 99.72%               | 99.93%              | 85.90%               | 86.10%              |
| <b>Fold 5 Avg. Accuracy</b>       | 94.99%               | 97.39%              | 95.97%               | 98.74%              |
| <b>Avg. Accuracy</b>              | 94.20%               | 95.10%              | 89.14%               | 90.94%              |
| <b>Standard Error</b>             | ± 4.98%              | ± 4.90%             | ± 6.14%              | ± 6.78%             |
| <b>Total Training Time (min.)</b> | 754                  | 540                 | 500                  | 352                 |



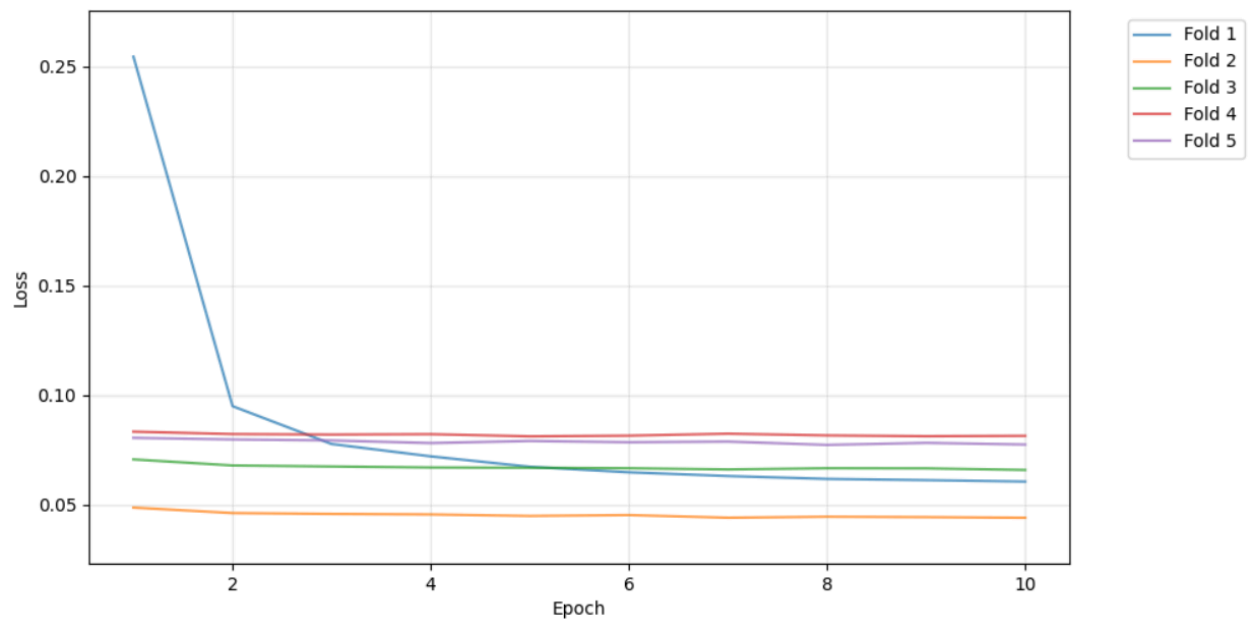


Figure 2: Simple\_VGG (K-Folds; round 1) - Training Loss per Fold

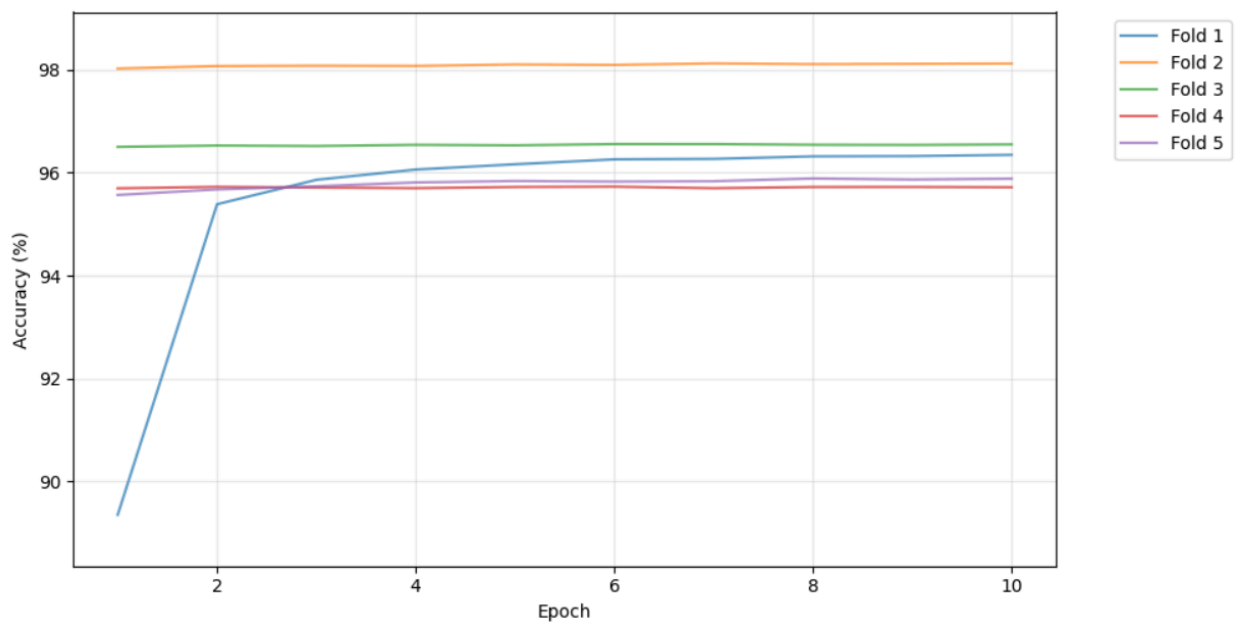


Figure 3: Simple\_VGG (K-Folds; round 1) - Training Accuracy per Fold

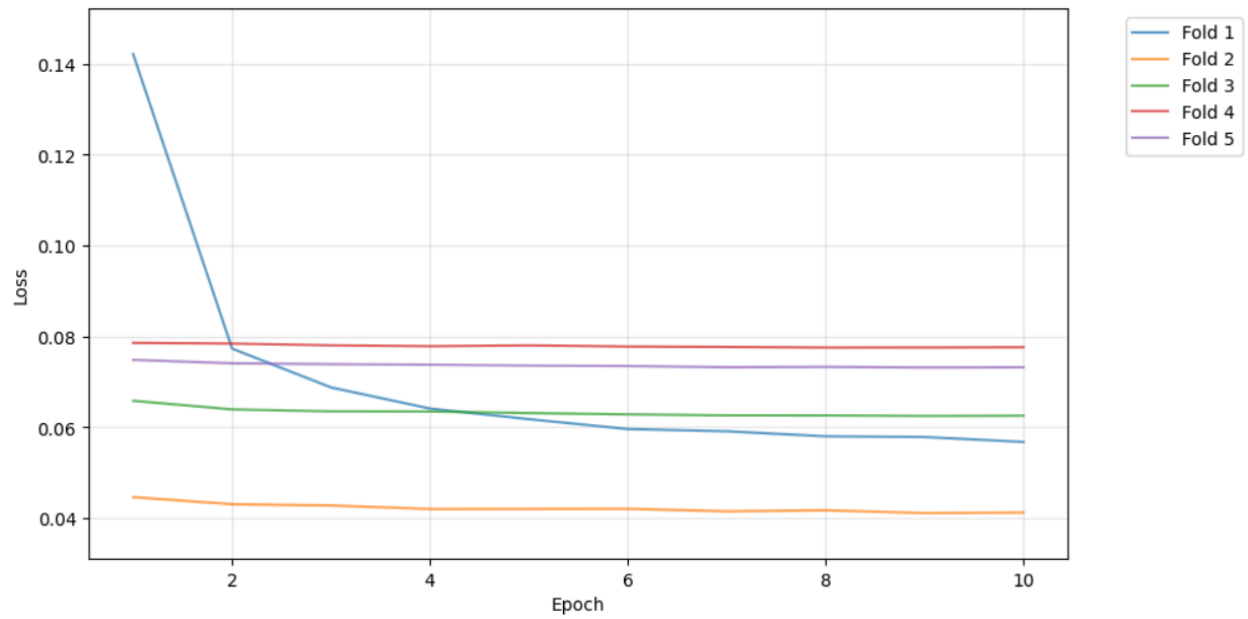


Figure 4: ResNet-18 (K-Folds; round 1) - Training Loss per Fold

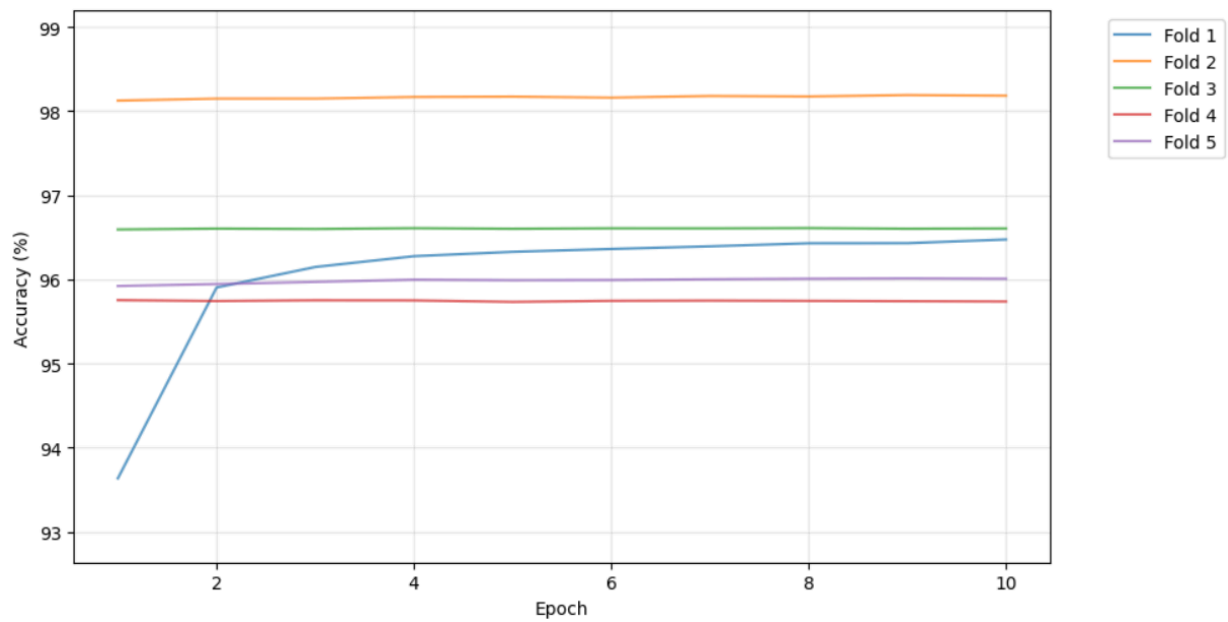


Figure 5: ResNet-18 (K-Folds; round 1) - Training Accuracy per Fold

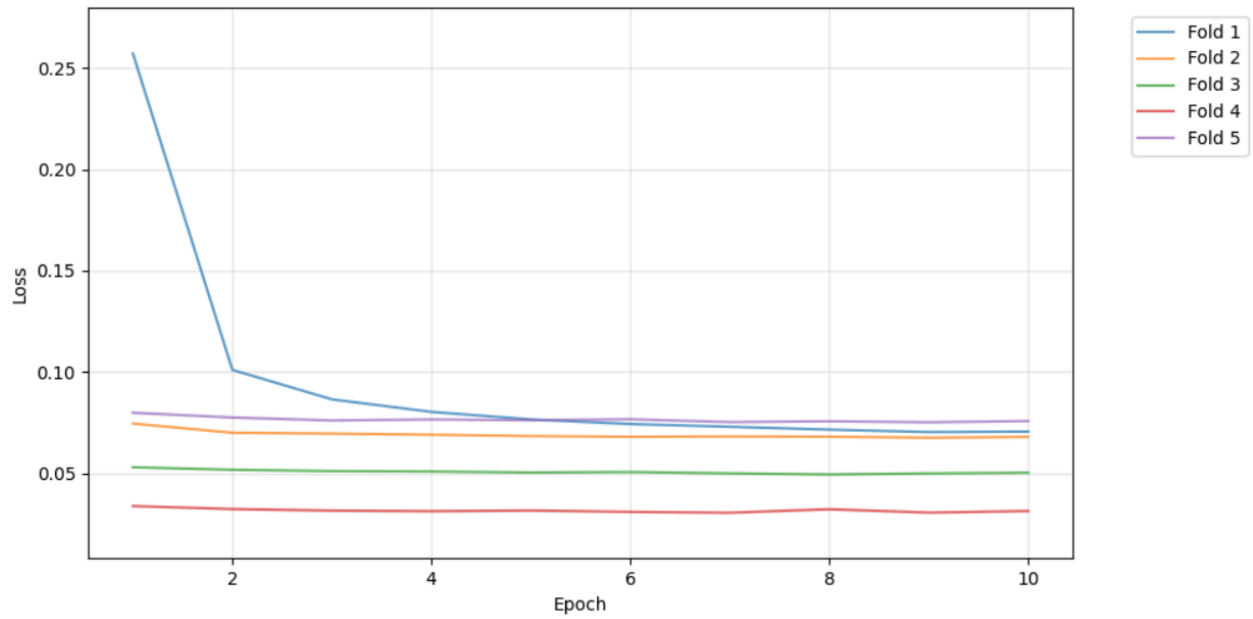


Figure 6: Simple\_VGG (K-Folds; round 2) - Training Loss per Fold

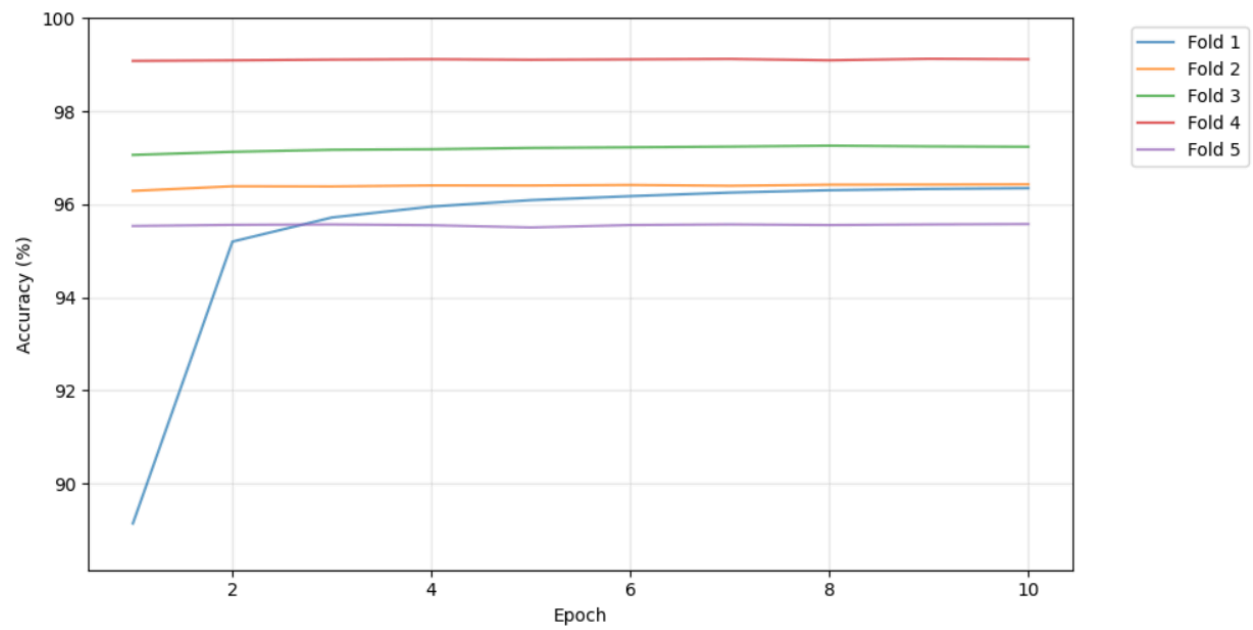


Figure 7: Simple\_VGG (K-Folds; round 2) - Training Accuracy per Fold

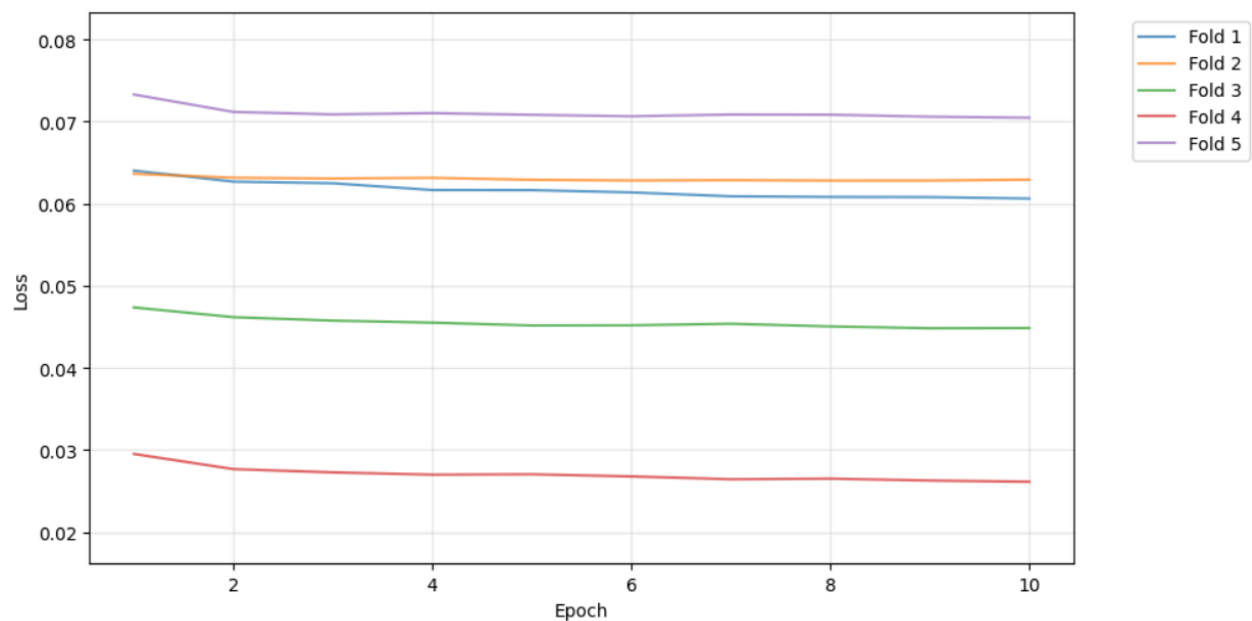


Figure 8: ResNet-18 (K-Folds; round 2) - Training Loss per Fold

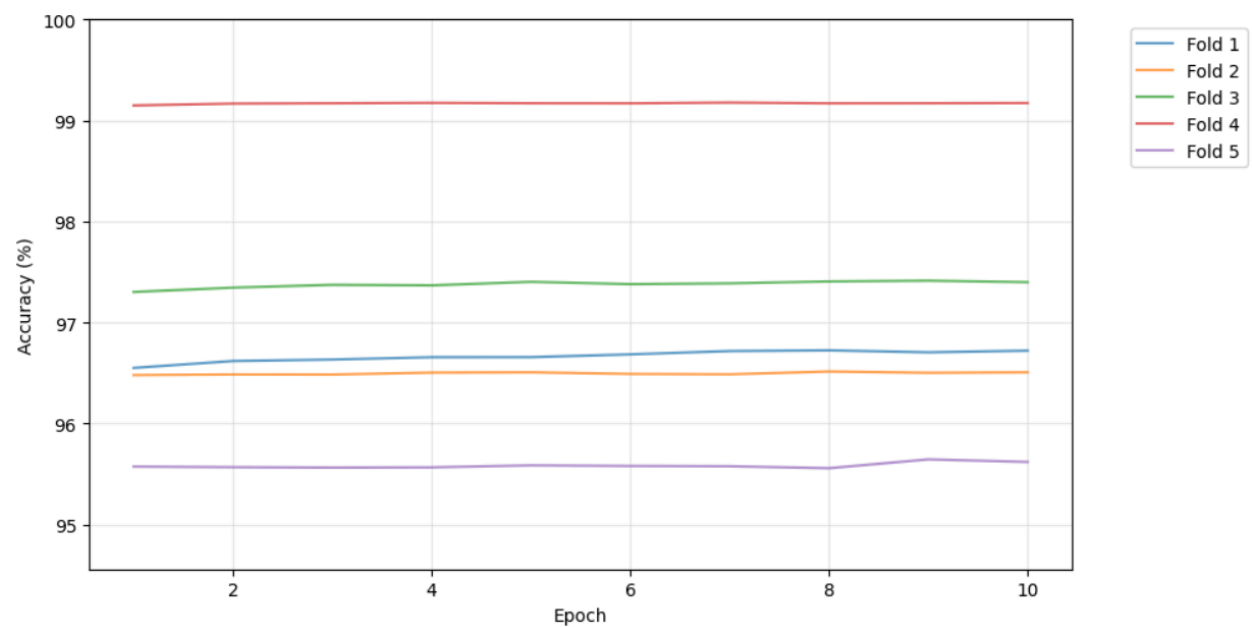
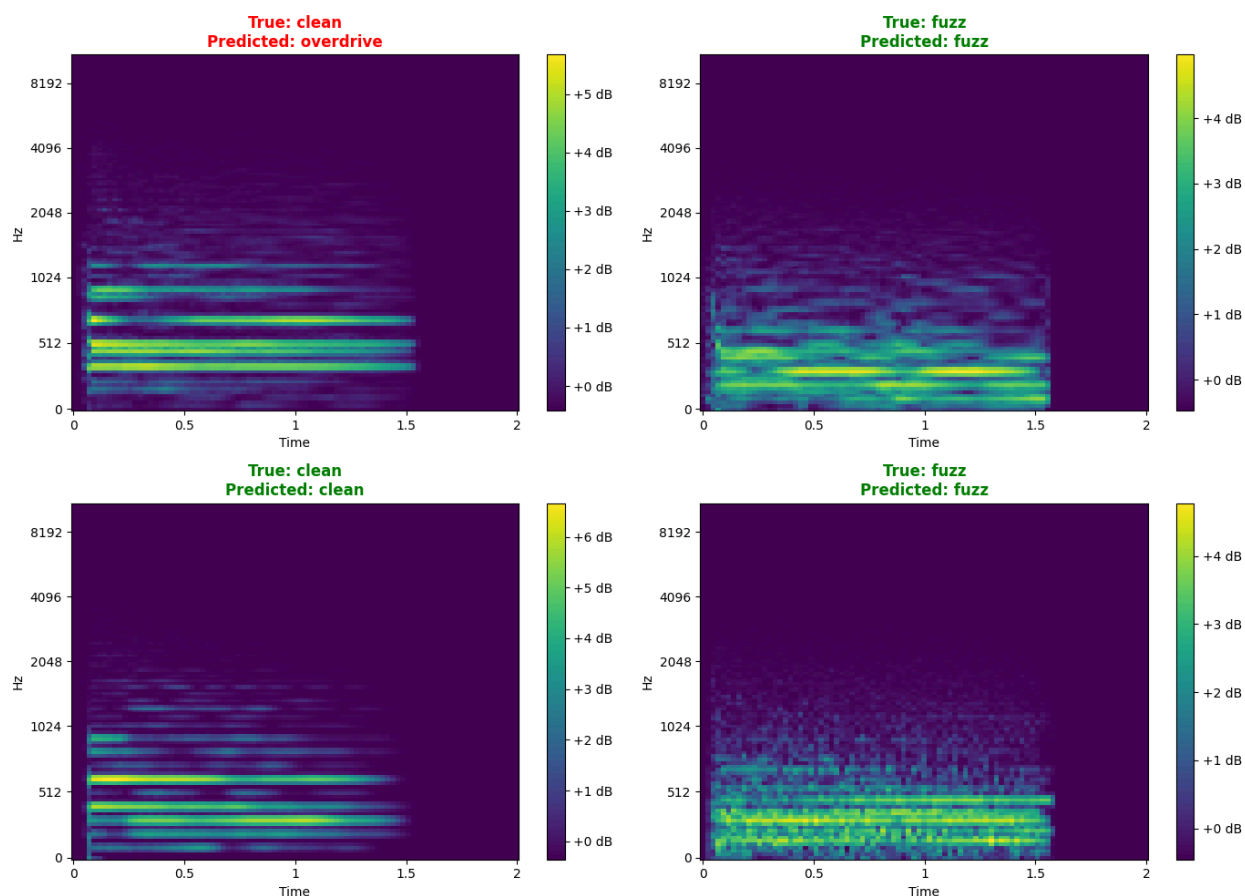


Figure 9: ResNet-18 (K-Folds; round 2) - Training Accuracy per Fold



**Figure 10: ResNet-18 (round 2) Example Predictions (Green indicates correctly predicted; Red indicates incorrect)**

Table 2 shows the validation results from the first training method implemented. Simple\_VGG Model 1 was trained over 3 rounds, with 10 epochs in each round, and achieved an overall accuracy of 94.94%. The training/test split was not changed across each round. This likely resulted in an overfitting of the data, given the general lack of increase in accuracy across each round. ResNet-18\_Pretrained was trained using a 90/10 train/test split across both of its rounds. The first round was run over 10 epochs, and the second round over 20 epochs. The train/test split was changed between the two rounds. The model achieved an overall accuracy of 99.3%. Due to the already high accuracy, an extra 10 epochs weren't run for this model.

Table 3 shows the results of the second training method I implemented using k-fold cross-validation. Each round represents a different k-fold test split that was used to train both models. Average accuracy and standard deviation are also provided for each round, as well as the total time it took to train the model. Note that the massive difference in times is a result of

my running the training with my computer set to quiet mode vs performance mode. Performance mode maximized the resources of my computer but made the fans super loud, while performance mode kept the fans as quiet as possible. Since I was running the simulations at night, sometimes I would change from performance to quiet mode; otherwise, I wasn't able to sleep. Figures 2-9 are plots for training loss and accuracy per fold for both models across both rounds. For most of the folds, both accuracy and loss stayed roughly the same across each epoch, with some notable differences that will be discussed next. Figure 10 is an example of a few mel-specs graphed out, with the predictions from the ResNet-18 model from round 2.

## Discussion and Conclusion

The results of this experiment are quite interesting. From both test methods, we can see that training both model architectures across multiple epochs generally doesn't result in a steady increase in accuracy, but rather remains consistent across. The notable exception is Fold 1 for round 2 of k-fold testing, which saw a sharp increase in accuracy followed by a flattening out, across both models. I'm not quite sure why this would be the case; it's possible that both models are really good at distinguishing between the different features of distortion, overdrive, and fuzz, and don't require as much training.

Furthermore, there doesn't seem to be a massive difference between using the simple\_VGG or ResNet-18 architecture. Both models performed roughly the same in the second training method, with ResNet-18 having a slightly higher overall accuracy. Both models have similar average accuracies across each fold, suggesting that the way the data was split had more of an impact on the final accuracy. The large differences in average accuracies across both models in both rounds of k-folds make sense; the data was split by number of HDF5 shards and not an exact number of mel-specs.

Even with these inconsistencies, it's clear that CNNs are a good choice for guitar effects classification. This is most highlighted by the fact that neither architecture achieved an average accuracy below 80% for any testing. Our highest accuracy being 99.3% accuracy is also a very good sign, though it's far less interpretable given that we don't know what the model was pretrained on.

Of course, this project is not without its limitations. Given the time constraints, testing was not very thorough. Future trials should be done using different hyperparameters to see how that affects accuracy. Based on the results, the method for generating training/test splits should be modified such that they're not done based on shards but rather have an even distribution of mel-specs across each category. This would likely solve the huge discrepancy between average accuracy across different folds.

The results of this project can only be extended to the overdrive, distortion, and fuzz guitar effects. Datasets that include other effects (e.g., chorus, reverb, phaser) would have to be generated and validated to make definitive conclusions as to the spectrum of effects that are able to be modeled. This project also never utilized samples longer than 2 seconds, which could heavily impact the applications of this model. Future datasets would need to consider different sample sizes, different combinations of effects at once, and multiple guitars being played at once.

Different models with different numbers of layers could have also been tested. One interesting idea would be to add more and more convolutional layers to the Simple\_VGG model and see how training loss and accuracy change over time across different numbers of epochs. Additionally, other model architectures could be explored to further evaluate if there are pros and cons across models when classifying guitar effects.

The ethical and social implications of this project are somewhat limited. Given that this deals with audio recordings, it would be possible to have a dataset that only contains recorded songs from public artists and/or commercially available songs. This could be problematic depending on how the music is acquired and whether or not the artist is ok with their work being used in machine learning-related projects. Similarly, if such a model is not trained across a variety of different instruments, styles, and combinations of effects from various cultures of music, then the model will be incredibly biased in what it can classify and will not be useful in the most general sense.

However, as showcased by the GFD dataset, through a lot of dedication and some scripting, it's possible to systematically create the data necessary to achieve a high level of accuracy. A model that could accurately classify any guitar pedal effect (or any sound for that

matter) would be very useful for beginner artists wanting to recreate their favorite artist's style and tone. Such a tool would be useful so that new artists who aren't as experienced don't have to spend loads of time trying to figure out a specific instrument and/or effect style, and more time can be spent on the creative uses for said effect. This type of model could also be very useful in archival work and audio engineering for organizing large audio libraries very quickly and systematically.

If I were to go back and redo this project, I would have done a few things differently. The first would have been to avoid using Google Drive at all. It is simply too slow to upload large data sets, its file system is not robust, and loading files from Drive to Colab is impractically slow. Many hours were spent simply trying to download and upload files in a consistent way that would be reproducible for others, but no such method was ever really found. In addition, I would have looked for a PC in my school to train the models on, rather than performing all of the training on my own machine. This would have made training go far faster, and more could've been run across days rather than just at night.

Overall, this project highlights the key struggles of most machine learning projects, namely, data collection and organization. It took a long time to get the datasets organized and loaded onto Colab just to train the first model, and that was even when it was already thoroughly processed and organized by the original authors. It also highlights just how much data is required to train such models, and the limitations that impose on those who are trying to do exploratory analysis of such topics, both in terms of the amount of time it takes to process, train, and validate the models, and also in the computational resources necessary for it. Nevertheless, I believe this project answers at least the first research question it set out to explore, and showcases what is possible in machine learning now, even when all you have is a single laptop.



## **Bibliography**

1. Bognár, P. (2022). Audio effect modeling with deep learning methods [Diploma Thesis, Technische Universität Wien]. repositUM. <https://doi.org/10.34726/hss.2022.94860>
2. Chulev, Joanikij. "Improving musical instrument classification with advanced machine learning techniques." arXiv preprint arXiv:2411.00275 (2024).  
<https://arxiv.org/abs/2411.00275>
3. de Castro Rabelo Profeta, R., Schuller, G. (2020). Feature-Based Classification of Electric Guitar Types. In: Cellier, P., Driessens, K. (eds) Machine Learning and Knowledge Discovery in Databases. ECML PKDD 2019. Communications in Computer and Information Science, vol 1168. Springer, Cham.  
[https://doi.org/10.1007/978-3-030-43887-6\\_41](https://doi.org/10.1007/978-3-030-43887-6_41)
4. F. Rong, "Audio Classification Method Based on Machine Learning," 2016 International Conference on Intelligent Transportation, Big Data & Smart City (ICITBS), Changsha, China, 2016, pp. 81-84, doi: 10.1109/ICITBS.2016.98. keywords: {Transportation;Big Data;Smart cities;Audio classification;Machine learning;SVM;Audio frame;Audio shot},  
<https://ieeexplore.ieee.org/abstract/document/8047110>
5. "Resnet18 from Scratch Using Pytorch." *GeeksforGeeks*, GeeksforGeeks, 27 Nov. 2025, [www.geeksforgeeks.org/deep-learning/resnet18-from-scratch-using-pytorch/](http://www.geeksforgeeks.org/deep-learning/resnet18-from-scratch-using-pytorch/).
6. Hinrichs, Reemt, et al. "Convolutional neural networks for the classification of guitar effects and extraction of the parameter settings of single and multi-guitar effects from instrument mixes." *EURASIP Journal on Audio, Speech, and Music Processing* 2022.1 (2022): 28.
7. Roberts, Leland. "Understanding the Mel Spectrogram ." *Medium*, Analytics Vidhya, 5 Mar. 2020,  
[medium.com/analytics-vidhya/understanding-the-mel-spectrogram-fca2afa2ce53](https://medium.com/analytics-vidhya/understanding-the-mel-spectrogram-fca2afa2ce53).

8. Stoner, Brandon. "Overdrive vs. Distortion vs. Fuzz: What's the Difference? Gaining a Clearer Understanding of Guitar Dirt Pedals." *PerfectCircuit*, 9 Sept. 2023, [www.perfectcircuit.com/signal/overdrive-distortion-fuzz-pedals](http://www.perfectcircuit.com/signal/overdrive-distortion-fuzz-pedals).
9. "VGG-Net Architecture Explained." *GeeksforGeeks*, GeeksforGeeks, 23 July 2025, [www.geeksforgeeks.org/computer-vision/vgg-net-architecture-explained/](http://www.geeksforgeeks.org/computer-vision/vgg-net-architecture-explained/).
10. Wasser, Leah A. "Hierarchical Data Formats - What Is HDF5?" NEON SCIENCE, 2 Apr. 2026, [www.neonscience.org/resources/learning-hub/tutorials/about-hdf5](http://www.neonscience.org/resources/learning-hub/tutorials/about-hdf5).
11. Velardo, Valerio. "Audio Signal Processing for Machine Learning." *YouTube*, YouTube, 18 June 2020, [www.youtube.com/playlist?list=PL-wATfeyAMNqlee7cH3q1bh4QJFAaeNv0](http://www.youtube.com/playlist?list=PL-wATfeyAMNqlee7cH3q1bh4QJFAaeNv0).

## **Datasets**

Marco Comunita'. GUITAR-FX-DIST: A Dataset of Processed Guitar Recordings for Music Research - (Mono Discrete). 1.0, Zenodo, 30 Nov. 2020, <https://doi.org/10.5281/zenodo.4298000>.

Kehling, C., et al. IDMT-SMT-Guitar Dataset. 1.0.0, Zenodo, 17 Jan. 2023, <https://doi.org/10.5281/zenodo.7544110>.